

操作系统第三次作业

(1) 读者写者问题 (写者优先)

```
1  int readers = 0;
2  Semaphore mutex = 1;
3  Semaphore roomEmpty = 1;
4  Semaphore writeBlock = 1; // 用于阻止新读者
5
6  Writer {
7      P(writeBlock); // 阻止新读者
8      P(roomEmpty); // 确保写者独占
9      write; // critical region
10     V(roomEmpty);
11     V(writeBlock); // 允许等待的读者或写者进入
12 }
13
14 Reader {
15     P(writeBlock); // 检查是否有写者等待
16     P(mutex); // 保护读者计数
17     readers = readers + 1;
18     if (readers == 1) {
19         P(roomEmpty); // 第一个读者, 阻止写者
20     }
21     V(mutex);
22     V(writeBlock); // 允许其他读者或写者检查
23
24     read; // critical region
25
26     P(mutex); // 保护读者计数
27     readers = readers - 1;
28     if (readers == 0) {
29         V(roomEmpty); // 最后一个读者, 允许写者
30     }
31     V(mutex);
32 }
33
```

关键在于 `writeBlock`，确保了有写者排队之后，阻塞所有读者的进入（锁住了 `readers` 增加的过程）。

(2) 寿司店问题

```
1  int seatsOccupied = 0; // 当前已占用的座位数
2  Semaphore mutex = 1; // 用于保护对座位数的访问
3  Semaphore allSeatsEmpty = 1; // 用于等待所有座位都空出来
4
5  Customer {
6      if (seatsOccupied < 5) {
7          // 如果有空座位, 顾客立即就坐
8          P(mutex);
9          seatsOccupied = seatsOccupied + 1;
10         if (seatsOccupied == 5) {
11             P(allSeatsEmpty);
12         }
13     }
14 }
```

```

12     }
13     V(mutex);
14     eat; // critical region
15     P(mutex);
16     seatsOccupied = seatsOccupied - 1;
17     if (seatsOccupied == 0) {
18         V(allSeatsEmpty);
19     }
20     V(mutex);
21 } else {
22     // 如果没有空座位，顾客等待所有座位都空出来
23     P(allSeatsEmpty);
24     // 顾客就坐
25     P(mutex);
26     seatsOccupied = seatsOccupied + 1;
27     V(mutex);
28     V(allSeatsEmpty);
29     eat; // critical region
30     P(mutex);
31     seatsOccupied = seatsOccupied - 1; // 这一组顾客离开，所有座位都空出
    来
32     if (seatsOccupied == 0) {
33         V(allSeatsEmpty);
34     }
35     V(mutex);
36 }
37 }
38

```

(3) 进门问题

- P、V的含义：

- P：检查信号量的值。如果信号量的值大于0，则将其减1，表示进程获得了资源。如果信号量的值为0，则进程被阻塞，直到信号量的值变为正数。
- V：它会将信号量的值加1，表示释放了一个资源。如果此时有其他进程因为等待这个信号量而被阻塞，那么其中一个进程会被唤醒。

- ```

1 int employeesArrived = 0; // 已刷卡员工数
2 int employeesEntered = 0; // 已进入员工数
3 Semaphore allArrived = 0; // 所有员工都已刷卡
4 Semaphore mutex = 1; // 保护对employeesArrived和employeesEntered的访问
5
6 Employee {
7 P(mutex);
8 employeesArrived = employeesArrived + 1;
9 if (employeesArrived == 5)
10 V(allArrived); // 最后一个员工刷卡，唤醒所有等待的员工
11 V(mutex);
12
13 P(allArrived); // 等待所有员工都刷卡
14 enterOffice; // 进入公司
15 P(mutex);
16 employeesEntered = employeesEntered + 1;
17 if (employeesEntered == 5) {
18 closeDoor; // 最后一个员工负责关门
19 }
20 V(mutex);

```

```

21 v(allArrived); // 允许下一个员工通过大门
22 }

```

#### (4) 搜索-插入-删除问题

```

1 int searchNum = 0;
2 Semaphore searchMutex = 1; // 控制搜索数量变量
3 Semaphore insertMutex = 1; // 控制插入操作的互斥
4 Semaphore deleteMutex = 1; // 控制删除操作的互斥
5 Semaphore searchDeleteMutex = 1; // 控制搜索和删除操作的互斥
6 Semaphore insertDeleteMutex = 1; // 控制插入和删除操作的互斥
7
8 // 搜索线程
9 Searcher {
10 P(searchMutex);
11 searchNum = searchNum + 1;
12 if (searchNum == 1) {
13 P(searchDeleteMutex); // 阻止删除操作
14 }
15 v(searchMutex);
16 searchList;
17 P(searchMutex);
18 searchNum = searchNum - 1;
19 if (searchNum == 0) {
20 v(searchDeleteMutex);
21 }
22 v(searchMutex);
23 }
24
25 // 插入线程
26 Inserter {
27 P(insertMutex);
28 P(insertDeleteMutex); // 阻止删除操作
29 insertIntoList;
30 v(insertDeleteMutex);
31 v(insertMutex);
32 }
33
34 // 删除线程
35 Deleter {
36 P(deleteMutex);
37 P(searchDeleteMutex);
38 P(insertDeleteMutex);
39 deleteFromList;
40 v(insertDeleteMutex);
41 v(searchDeleteMutex);
42 v(deleteMutex);
43 }

```

- 搜索线程：
  - 当有删除线程工作时，`searchDeleteMutex` 被删除线程持有，无法进入。
  - 当无删除线程工作时，可以进入，维护 `searchNum`。第一个搜索线程进入时，获取 `searchDeleteMutex`，阻止删除线程；最后一个搜索线程离开时，释放 `searchDeleteMutex`。
- 插入线程：

- 当有删除线程工作时, `insertDeleteMutex` 被删除线程持有, 无法进入。
- 当有其他插入线程工作时, `insertMutex` 被其他插入线程持有, 无法进入。
- 当无删除线程工作且无其他插入线程工作时, 可以进入, 获取 `insertDeleteMutex` 和 `insertMutex`, 离开时释放。
- 删除线程:
  - 当有搜索线程工作时, `searchDeleteMutex` 被持有, 无法进入。
  - 当有插入线程工作时, `insertDeleteMutex` 被持有, 无法进入。
  - 当其他删除线程工作时, `deleteMutex` 被持有, 无法进入。
  - 当无其他线程工作时, 可以进入, 获取以上三个锁。